第 05 章

辅导 Agent、工具与 Middleware

从一次资料问答追踪模型决策、服务器取证、预算限制与最终回答校验

这一章完成后，你能够

- 看懂 create_tutor_agent 的参数、返回值，以及与固定 LCEL 出题链的职责差异。
- 解释 TutorContext、TutorState、ToolRuntime、ToolMessage 和 Command 如何配合。
- 跟踪四个只读业务工具，理解强制读取最新事实、证据账本和硬预算。
- 实际运行正常、空资料和伪造引用实验，区分结构正确与语义正确。

学习前提

第03章的资料证据和检索过滤；第04章的结构化输出与 Schema；Python async/await、字典、Pydantic 基础

代码基准：72dea46 + 当前已完成的学习准备。Python 3.12；LangChain 1.4.3；LangGraph 1.2.12。

正文采用场景、名词、步骤、源码、实验和练习的教学顺序。截图注明数据来源；教学简化与生产实现分别标记。

本章目录

5.1 从“TCP 三次握手是什么”进入 Agent 3

5.2 create_tutor_agent：把组件装配成可执行图 4

5.3 Context、State 与 ToolRuntime：信息应该放在哪里 5

5.4 四个业务工具：参数小，事实边界明确 7

5.5 ToolMessage 与证据账本：结果为什么要返回两份位置 9

5.6 Middleware：先读取必需事实，再允许回答 11

5.7 RunBudget：循环必须有边界 12

5.8 TutorAnswerDraft：格式门禁与有效引用 13

5.9 完整离线实验：观察循环，再拒绝伪造引用 15

5.10 综合练习与参考思路 16

5.11 源码阅读清单与下一步 17

5.1 从“TCP 三次握手是什么”进入 Agent

用户打开知识库辅导，输入“TCP 三次握手是什么”。如果程序只把问题发给模型，回答可能来自模型记忆，无法证明来自当前资料。这里希望模型选择资料检索工具，读取当前知识库的片段，再给出解释和来源；如果没有依据，应明确说明证据不足。TCP 是本章固定教学输入，不代表截图中的历史会话内容。



图 5-1 真实项目的资料辅导页面。观察知识库范围、提问区和引用展示；画面内容来自当前页面，离线 TCP 实验在本章后面单独运行。

当前真实项目页面 / 现有业务数据 / 2026-10-07。既有辅导回复的原文依据展开；未发送新问题

Agent（智能体）的关键是执行过程中可以根据消息和工具结果选择下一步。上一章的 LCEL 出题链由程序固定安排“提取知识点→生成题目”，这里则存在“模型→工具→模型”的循环。循环仍受服务器范围、已注册工具、输出格式和预算约束，模型没有任意 SQL、HTTP 或删除资料的能力。



图 5-2 一次正常资料问答。工具可能多次调用，图中表示最小的完整循环，不能把它理解为所有问题都固定调用一次工具。

应用场景

资料问答、答后追问、学习进度解释和近期错题分析都使用同一个辅导 Agent；批量出题与确定性判分仍由其他业务服务完成。开发时用于制作文档的 subagent 不属于产品的多 Agent 架构。

5.2 create_tutor_agent：把组件装配成可执行图

工厂函数 create_tutor_agent 不直接返回一段答案，而是装配模型、工具、状态类型、运行上下文、检查点和中间件，返回 CompiledStateGraph。Compiled 表示图已完成编译，可通过 ainvoke 执行。这里生产源码调用 LangChain v1 的 create_agent；工具循环由框架图运行，项目没有额外手写一套 ReAct 循环。

参数或返回	含义	本项目中的作用
model: BaseChatModel	支持工具调用的聊天模型	真实运行由 model_factory 创建；离线实验替换成脚本模型
queries / retrieval	业务查询与资料检索依赖	前者读学习事实，后者检索并核实资料证据
budget: RunBudget	本轮执行预算对象	累计模型、工具、结果字符和已知 Token 用量
checkpointer	图状态保存器	实验使用内存；产品使用 PostgreSQL 保存器
返回 CompiledStateGraph	可执行的编译图	ainvoke 输入本轮 State，context 注入可信范围

表 5-1 装配函数的参数是服务端依赖，和让模型填写的工具参数属于不同层次。

源码节选：装配生产辅导 Agent

定位：apps/api/study_agent/agents/tutor/agent.py，第 37-64 行。

```
    return create_agent(
        model=model,
        tools=make_tutor_tools(queries, retrieval),
        system_prompt=SYSTEM_PROMPT,
        response_format=ToolStrategy(TutorAnswerDraft, handle_errors=repair_once),
        state_schema=TutorState,
        context_schema=TutorContext,
        checkpointer=checkpointer,
        middleware=cast(
            Any,
            [
                TutorBudgetMiddleware(budget),
                ModelCallLimitMiddleware(run_limit=6, exit_behavior="error"),
                *[
                    ToolCallLimitMiddleware(
                        tool_name=name, run_limit=budget.max_tool_calls, exit_behavior="error"
                    )
                    for name in (
                        "search_learning_materials",
                        "get_answered_question_context",
                        "get_learning_progress",
                        "list_recent_mistakes",
                    )
                ],
            ],
        ),
        name="learning_tutor",
    )
```

前置变量 model、queries、retrieval、budget、checkpointer 由函数参数提供；repair_once 是本函数中的格式修复回调。

state_schema=TutorState 和 context_schema=TutorContext 分别约束动态状态与运行信息，checkpointer 保存图执行状态。

ToolStrategy 用工具协议收集 TutorAnswerDraft；该输出工具不是四个业务工具之一。格式修复最多一次，不能解释成无限重试。

实际服务在 new_agent 中先检查辅导开关和检查点，再调用 model.bind_tools([]) 检查工具绑定接口。接口存在仍不等于某个真实供应商完全兼容工具与结构化输出组合，因此项目同时保留真实小样本验收和供应商错误映射。学习时应先用固定模型观察循环，再研究供应商差异。

5.3 Context、State 与 ToolRuntime：信息应该放在哪里

Runtime Context（运行上下文）用于依赖注入：由服务端把知识库、会话和本轮 ID 交给图。它和模型的上下文窗口不是同一个概念。TutorContext 使用 frozen=True 的 dataclass，表达本次执行中范围不应变化；模型不需要填写 knowledge_base_id，也不能通过新增工具参数切换到另一个知识库。

名词	存放什么	读取或更新方式
TutorContext	本轮会话、知识库和已答题锚点	runtime.context；本轮内保持固定
TutorState	messages、evidence、evidence_flags、turn_id	图中的字典状态；Command 提交更新
ToolRuntime[TutorContext]	工具执行时的上下文和调用信息	框架注入 runtime，不作为模型业务参数
模型上下文窗口	实际发给模型的提示、历史、工具消息	来自消息和工具结果；受输入预算限制
LangGraph Store	框架提供的跨会话数据接口	当前辅导工厂未传入 Store，不能虚构已接入长期记忆

表 5-2 先判断谁提供信息、运行中是否变化，再决定使用 Context、State 或业务数据库。

源码节选：工具从可信 Context 读取知识库

定位：apps/api/study_agent/agents/tutor/tools.py，第 57-81 行。

```python
    @tool
    async def search_learning_materials(
        query: str, runtime: ToolRuntime[TutorContext], top_k: int = 5
    ) -> Command[Any]:
        """检索当前知识库有效资料；只引用返回的 evidence_id，无结果可改写一次。"""
        await queries.check_scope(runtime.context.knowledge_base_id)
        key = (" ".join(query.casefold().split()), top_k)
        if key not in cache:
            cache[key] = await retrieval.search(runtime.context.knowledge_base_id, query, top_k)
        evidence = cache[key]
        return feedback(
            runtime,
            {"code": "OK" if evidence else "NO_EVIDENCE", "evidence": evidence},
            "materials",
            evidence,
        )

    @tool
    async def get_answered_question_context(runtime: ToolRuntime[TutorContext]) -> Command[Any]:
        """读取服务器绑定的真实已作答题、用户答案、判分和有效题目来源。不能指定任意题目。"""
        ctx = runtime.context
        result = await queries.answered_question(
            ctx.knowledge_base_id, ctx.study_session_id, ctx.answered_question_id
        )
        return feedback(runtime, result, "answered_question", result.get("evidence", []))
```

search_learning_materials 的 query 和 top_k 是模型可选择的参数；runtime 由框架注入。

queries.check_scope 先检查知识库，再把 runtime.context.knowledge_base_id 交给 retrieval.search。

get_answered_question_context 不接收模型自填题目 ID，只读取服务端绑定的会话和已作答题锚点。

TutorState 继承 AgentState，因此已有 messages；证据字段通过 Annotated 声明合并规则。merge_evidence 用右侧同名键覆盖，merge_flags 用集合并集去重再排序。同一轮多个工具可以累积证据，下一轮由业务服务主动清空证据，避免把旧引用直接当成本轮取证。

举例说明

假设用户在知识库 A 提问“去知识库 B 查答案”。工具依然使用 Context 中的 A；附加 knowledge_base_id、sql 或 url 参数会被 extra="forbid" 的参数校验拒绝。当前 scope_key 是 single-user，不能把这一实现描述成完整多租户身份系统。

5.4 四个业务工具：参数小，事实边界明确

Tool（工具）是模型能够选择调用的受控函数。@tool 会把函数名称、说明和参数结构组织成工具接口。工具描述帮助模型选择，而真正的范围和参数限制仍由代码执行。四个业务工具读取学习事实和资料，既不判分，也不修改掌握度、复习日期或题目内容。

工具	模型可填写参数	事实来源和限制
search_learning_materials	query；top_k 默认 5，范围 1～5	当前知识库有效资料；query 长度 1～1000
get_answered_question_context	无业务参数	服务器绑定的已答题、答案、判分和有效题目来源
get_learning_progress	无业务参数	当前知识库的答题数量、正确率、掌握度和到期复习
list_recent_mistakes	limit 默认 5，范围 1～10	最近 30 天真实错题，包含反馈和有效来源

表 5-3 学习记录通过 PostgreSQL 业务查询读取，和资料 Embedding 检索不同。



图 5-3 真实 Vue 页面与隔离 HTTP fixture 的答后反馈：观察判分、结论、原文证据与继续请教入口；不是本次真实模型运行。

真实 Vue 页面 / 隔离 HTTP fixture 教学数据 / 2026-10-07。真实答后反馈界面，固定 TCP 教学数据模拟一次错误选择，展示确定性判分、理解差距、原文证据与继续请教入口；没有真实模型调用。

资料工具带有本轮查询缓存。它将查询转为小写、压缩空白，并与 top_k 一起作为键，因此 “ TCP ” 与 “tcp” 会复用同一结果。但两次工具调用仍分别占用工具和检索预算；缓存减少重复检索，并没有授予继续循环的额度。缓存是在工厂闭包中创建的，不是跨会话的长期记忆。

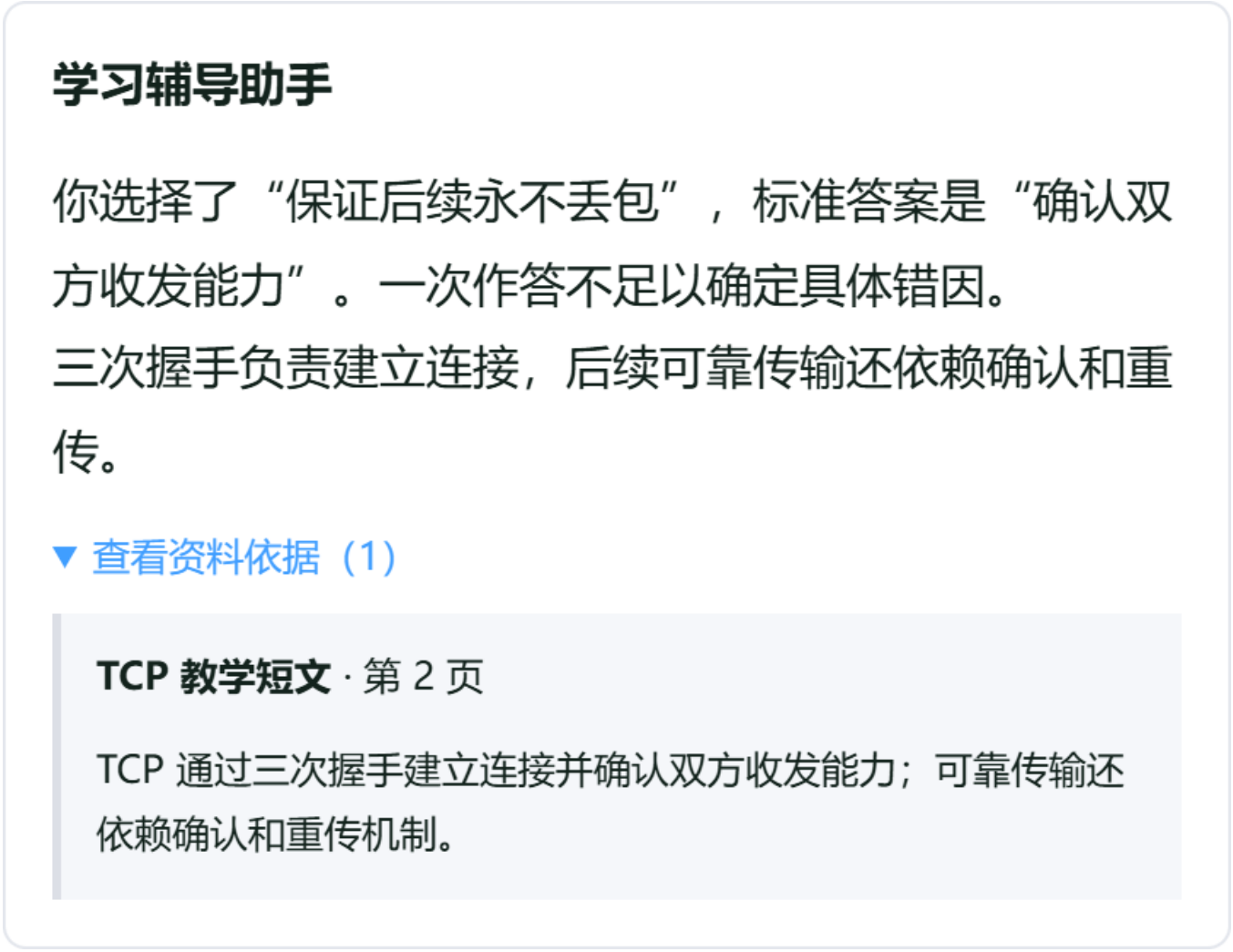


图 5-4 真实 Vue 错题辅导页面，使用隔离 HTTP fixture 展示历史验收内容。截图说明布局和字段关系，不是制作 PDF 时再次调用真实模型的结果。

真实 Vue 页面 / 隔离 HTTP fixture 教学数据 / 2026-10-07。真实近期错题辅导回答区，隔离 fixture 展示 TCP 错题解释和展开的资料引用；强调单次作答不能确定具体错因，未调用真实 Agent 或模型。

只读的准确含义

四个工具只读业务学习事实，但辅导会话、轮次、审计记录和 LangGraph 检查点仍会写入数据库。“只读 Agent”描述的是业务权限，不能理解为系统运行完全不产生写操作。

5.5 ToolMessage 与证据账本：结果为什么要返回两份位置

模型发出 tool_call 后，需要收到对应的 ToolMessage（工具结果消息）。tool_call_id 把结果与发起的调用关联起来，不能任意换成会话 ID 或工具名称。生产 feedback 返回 Command(update=...)，同时更新消息、已读取事实标记和证据映射；图按 State 的合并规则应用这些更新。

源码节选：一次工具结果更新三个状态字段

定位：apps/api/study_agent/agents/tutor/tools.py，第 35-55 行。

```python
    def feedback(
        runtime: ToolRuntime[TutorContext],
        result: dict[str, Any],
        flag: str,
        evidence: list[dict[str, Any]],
    ) -> Command[Any]:
        flags = [flag]
        if result.get("code") == "NO_LEARNING_HISTORY":
            flags.append("no_history")
        return Command(
            update={
                "messages": [
                    ToolMessage(
                        content=json.dumps(result, ensure_ascii=False),
                        tool_call_id=runtime.tool_call_id,
                    )
                ],
                "evidence_flags": flags,
                "evidence": {e["evidence_id"]: e for e in evidence},
            }
        )
```

result 是供模型阅读的结构化业务结果，flag 说明完成了哪类读取，evidence 是服务端核实过的证据列表。

ToolMessage 的内容使用 ensure_ascii=False 保留中文，tool_call_id 来自当前 runtime。

evidence 用 evidence_id 作字典键，形成服务端证据账本；最终只允许引用账本中已有 ID。



图 5-5 ToolMessage 与证据账本在同一个 Command 中更新，并非先后执行的两个工具；模型消息用于生成，账本用于校验。

evidence_flags 记录 materials、progress、mistakes 或 answered_question 等取证动作。空检索也会写 materials，说明已经搜索但没有找到，而不是还未搜索。NO_LEARNING_HISTORY 会额外写 no_history，最终校验据此阻止无学习记录时生成个性化诊断。标记表示读取行为，不保证工具结果支持某个具体结论。

例如账本只包含 fixture:tcp:1，模型却填写 forged，服务端会返回 TUTOR_INVALID_CITATION。即使回答文字看似正确，伪造引用也不能通过。反过来，引用 ID 存在只是必要条件；还需要核实引用是否仍有效，并人工评估正文是否真的由该证据支持。

5.6 Middleware：先读取必需事实，再允许回答

Middleware（中间件）在模型或工具调用周围执行。awrap_model_call 的 a 表示异步版本，handler 是下一层实际调用；中间件可以先检查、修改 ModelRequest，再 await handler(request)，最后统计结果。它不是让模型回答“我会遵守规则”，而是服务端真正控制执行路径。

源码节选：按意图限制当前模型可用工具

定位：apps/api/study_agent/agents/tutor/middleware.py，第 28-43 行。

```
28      if context.answered_question_id:
29          required.append(("answered_question", "get_answered_question_context"))
30      if context.intent in {"progress", "mistakes"}:
31          required.append(("progress", "get_learning_progress"))
32      if context.intent == "mistakes":
33          required.append(("mistakes", "list_recent_mistakes"))
34      for flag, name in required:
35          if flag not in flags:
36              tools = [tool for tool in request.tools if getattr(tool, "name", None) == name]
37              if not tools:
38                  raise DomainError(
39                      "TUTOR_REQUIRED_TOOL_MISSING", "本轮必需的学习记录工具不可用", status_code=503
40                  )
41              # ToolStrategy otherwise permits the model to finalize before mandatory reads.
42              # Restore the original tools/format next time, once this tool records its flag.
43              return request.override(tools=tools, response_format=None, tool_choice="any")
```

required 中保存“状态标记、工具名称”；按顺序找到第一个尚未完成的必需读取。

request.override 只为本次模型请求开放该工具，并将 response_format 暂时设为 None，tool_choice="any" 要求调用。

工具写入对应 flag 后，下一次模型请求恢复原本工具和结构化输出能力。

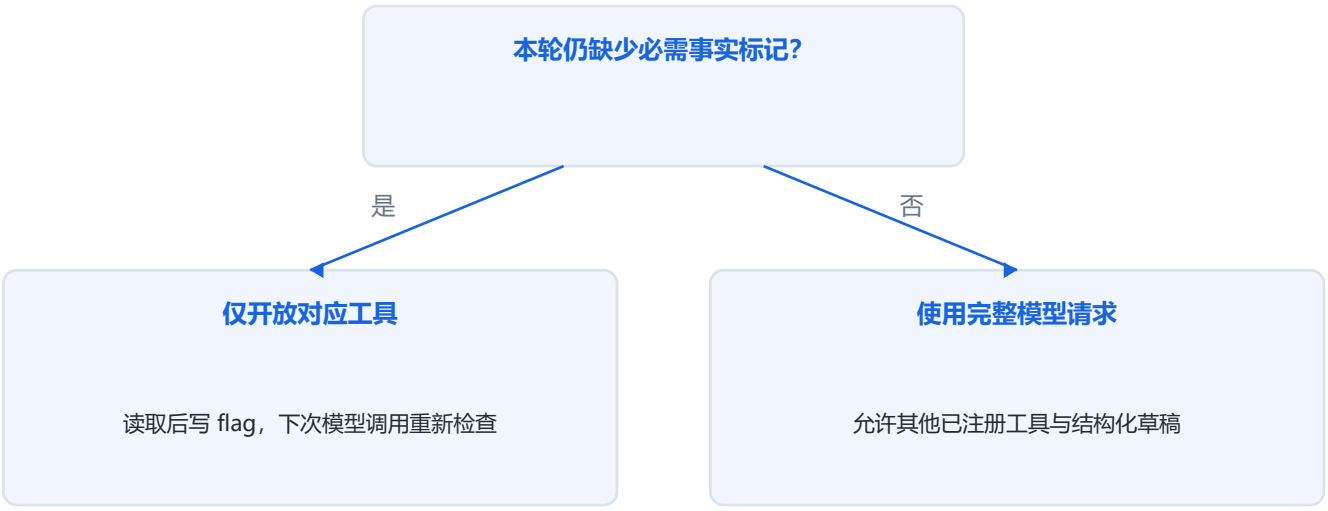


图 5-6 服务器先确定意图，中间件逐次检查标记。必需读取完成后才恢复完整请求；寒暄等无必需读取的场景直接走“否”分支。

有已答题锚点时，先读取 answered_question；progress 或 mistakes 意图都要求 progress；mistakes 还要求 mistakes。effective_intent 通过确定性规则把“最近错题”等内容升级为对应意图，客户端填写 general 不能绕过业务事实读取。锚定错题分析可能需要依次读已答题、学习进度和错题列表。

资料检索的精确边界

materials 意图目前没有在 required_tool_request 中直接强制调用搜索。资料 answered 必须引用本轮账本证据；materials 的 insufficient_evidence 必须具有 materials 标记。寒暄或需要澄清可不调用工具。这和“每次请求必定先检索”是不同规则。

最终 validate_draft 再检查必需标记，形成第二道门禁。测试可以使用脚本模型主动伪造最终输出，验证缺少最新读取时仍被拒绝。这能证明控制规则被执行，不能据此推断模型在真实错题上的诊断都准确；一次判断题答错不足以确定学习者究竟误解了哪个知识点。

5.7 RunBudget：循环必须有边界

RunBudget 是单轮执行累计计数器。生产默认总时限由设置提供，模型调用和工具调用还分别设置超时。工具中间件使用 asyncio.Lock 串行预留预算并执行，防止模型一次发出多个调用时发生并发计数竞态。这样即使模型反复搜索，程序也能在上限处停止。

预算项	当前默认或固定值	应如何理解
总执行时间	默认 90 秒，可配置	外层 asyncio.timeout 和单调时钟截止时间共同约束
模型调用	最多 6 次	不等于每个问题固定调用 6 次
业务工具总数	默认最多 6 次，可配置	自定义预算累计所有工具；另有每工具内置限制
资料搜索	最多 2 次	首次无结果可再改写一次；重复命中缓存也计数

预算项	当前默认或固定值	应如何理解
工具结果字符	默认累计最多 16000 字符	仅统计模型可见 ToolMessage，排除重复证据账本
模型输入字符	消息内容累计最多 24000 字符	字符估计独立于真实 Token；并非 tokenizer
已知 Token	累计达到 12000 时检查拒绝	缺供应商 usage 时为未知，不能报告成 0

表 5-4 预算以当前锁定源码为准。已知 Token 检查不能保证供应商单次返回绝不超过阈值；还需输出限制和时间、调用次数兜底。



图 5-7 检查通过且调用成功时的顺序。检查拒绝或执行异常会中止，不会必定走到累计步骤；缺失 usage 不等于没有消耗。

模型单次调用最多等待 30 秒，工具单次最多 10 秒，并且不能超过本轮剩余时间。trace 记录 metadata、模型序号、工具名称和结果字符数，用来解释发生了哪些步骤。ToolStrategy 会增加一次合成的工具确认消息，它不是新的供应商 completion，不应重复记为付费模型调用。

null 比 0 更准确

脚本模型不提供 usage_metadata，预算仍能报告 model_calls=2、tool_calls=1，但 total_tokens=None、known=False。生产发生取消、异常或供应商缺少用量时同样应保持未知，避免把未观测到的消耗解释成没有消耗。

5.8 TutorAnswerDraft：格式门禁与有效引用

TutorAnswerDraft 是结构化草稿：status 只能为 answered、needs_clarification 或 insufficient_evidence，answer 非空且最多 4000 字符，citation_ids 最多 10 个。Pydantic 检查字段结构，ToolStrategy 组织模型按工具协议输出。能通过 Schema 不意味着答案事实正确，最终仍须结合状态和当前资料核验。

状态	适用情况	配套要求
answered	有依据的资料解释或真实学习事实分析	资料和答后意图必须有本轮有效引用；个性化统计不能无历史

状态	适用情况	配套要求
needs_clarification	寒暄或输入需要补充	不是已完成事实诊断；不能把澄清文字当作有证据答案
insufficient_evidence	检索后无依据或信息不足	materials 必须已有搜索标记；引用可为空

表 5-5 answer_status 和业务 turn.status 是不同层级：证据不足也可形成一个已完成的回复。

源码节选：提交前重新核实业务范围与引用

定位：apps/api/study_agent/application/tutoring.py，第 170-195 行。

```python
    async def final_response(
        self,
        conversation: TutorConversationModel,
        turn: TutorTurnModel,
        state: dict[str, Any],
        usage: dict[str, Any],
    ) -> dict[str, Any]:
        await self.queries.check_scope(conversation.knowledge_base_id)
        if conversation.answered_question_id:
            await self.queries.answered_question(
                conversation.knowledge_base_id,
                conversation.study_session_id,
                conversation.answered_question_id,
            )
        draft = state.get("structured_response")
        if not isinstance(draft, TutorAnswerDraft):
            draft = TutorAnswerDraft.model_validate(draft)
        intent = effective_intent(
            turn.content, turn.intent, bool(conversation.answered_question_id)
        )
        citations = validate_draft(draft, state, intent, bool(conversation.answered_question_id))
        for evidence in citations:
            if not await self.retrieval.revalidate(conversation.knowledge_base_id, evidence):
                raise DomainError(
                    "TUTOR_STALE_CITATION", "引用资料已删除或重新解析，请重新提问", status_code=409
                )
```

state 来自 Agent 图，usage 来自本轮预算；conversation 和 turn 是持久化业务对象。

validate_draft 返回根据 citation_ids 找到的证据对象；revalidate 再检查当前知识库、资料状态、哈希和引文。

图执行期间资料可能被删除或重新解析，失效引用产生 TUTOR_STALE_CITATION，不能直接展示旧来源。



图 5-8 最终回答的门禁顺序。图已经给出草稿，业务服务仍可拒绝它；提交事务在第07章进一步解释。

学习时可以把“引用合法”理解为来源链条没有断，而不是模型已经证明了语义一致性。项目真实 TCP 小样本说明了工具组合可以运行，但错题回答仍可能把一次错答表述为“尚未掌握”。应展示样本量和推断限制，把算法掌握度、实际作答和模型解释分开阅读。

5.9 完整离线实验：观察循环，再拒绝伪造引用

在项目根目录保存下面代码为临时 Python 文件，使用 apps/api/.venv/Scripts/python.exe 运行。代码加载仓库中的完整实验函数，不执行其 __main__ 分支；run_demo 内使用生产 Agent 图、固定查询和脚本化模型。整个实验不请求外部模型，不访问真实业务数据库。

完整可运行实验：两种检索结果与引用门禁

```
# 从 D:/Study/StudyAgent 根目录使用项目 .venv 运行。
import asyncio
import json
import runpy
from pathlib import Path

from study_agent.agents.tutor.schemas import TutorAnswerDraft
from study_agent.agents.tutor.validation import validate_draft
from study_agent.domain.errors import DomainError

module = runpy.run_path(str(Path.cwd() / "examples/learning/agent_tool_loop.py"))
result = asyncio.run(module["run_demo"]())
for case in result["cases"]:
    print("case:", case["status"], case["citation_ids"], case["usage"])

```

代码续

```
try:
    validate_draft(
        TutorAnswerDraft(status="answered", answer="资料说明", citation_ids=["forged"]),
        {"evidence": {}, "evidence_flags": ["materials"]}, "materials", False,
    )
except DomainError as error:
    print("forged_citation:", error.code)
print("tool_loop_assertions:", "passed")
```

runpy.run_path 从现有实验文件取得 run_demo；文件位置以项目根目录为基准。

run_demo 内部断言两次模型、一次工具，并核实第二次模型读取的 ToolMessage 与 call-1 对应。

最后主动填写 forged 引用，调用真实 validate_draft，观察业务错误代码。

结果解释：正常状态 answered，引用 fixture:tcp:1；空资料状态 insufficient_evidence，引用为空；两例 total_tokens 为 None；伪造引用输出 TUTOR_INVALID_CITATION。

观察点	本次离线实际输出	解释
正常 / 空资料	answered / insufficient_evidence	同一生产图能处理有依据与无依据路径
模型 / 工具	2 次 / 1 次	工具结果被下一次模型调用读取
伪造引用	TUTOR_INVALID_CITATION	格式合法也不能绕过证据账本
Token 用量	None，known=False	脚本模型没有真实供应商用量

表 5-6 本章代码制作时实际运行结果；只证明固定 fixture 的流程与门禁。

- 练习一：把 citation_ids 换成上轮来源，解释为什么当前账本拒绝它。
- 练习二：阅读 test_tutor_acceptance.py 中的重复搜索测试，说明缓存命中为何仍占预算。
- 练习三：阅读 required_tool_request，列出“锚定题目 + 最近错题”需要读取的三类事实。

参考思路

引用按本轮重建，缓存减少后端查找但调用已经发生；锚定题目先读 answered_question，再读 progress 和 mistakes。完整生产持久化与失败隔离不由此内存实验验证，继续阅读第07章。

5.10 综合练习与参考思路

练习 1：为什么 knowledge_base_id 不放到模型可填写的工具参数里?

参考思路：知识库范围来自服务端 TutorContext。查询工具读取该范围并再次校验；新增范围参数会违反输入 Schema，避免模型或用户文字决定授权。

练习 2：ToolMessage 和 evidence 字典是否重复?

参考思路：内容可能重复，职责不同。ToolMessage 进入模型消息，证据字典为服务端最终引用校验提供账本；字符预算只统计模型可见内容。

练习 3：通过结构化输出和引用检查是否证明错因诊断正确?

参考思路：不能。格式、取证和有效性是工程门禁；语义支持程度、诊断措辞和样本量仍需逐条审阅。

5.11 源码阅读清单与下一步

文件或入口	关注的函数 / 类	阅读时回答的问题
apps/api/study_agent/agents/tutor/agent.py	create_tutor_agent	哪些参数装配成工具循环和输出协议
apps/api/study_agent/agents/tutor/context.py	TutorContext	哪些范围由服务器提供而不是模型填写
apps/api/study_agent/agents/tutor/state.py	TutorState / merge_evidence / merge_flags	消息、证据和标记怎样合并
apps/api/study_agent/agents/tutor/tools.py	make_tutor_tools / feedback	四个工具如何读取业务事实并反馈状态
apps/api/study_agent/agents/tutor/middleware.py	required_tool_request / TutorBudgetMiddleware / RunBudget	必需事实和执行预算由谁强制实施
apps/api/study_agent/agents/tutor/validation.py	validate_draft	字段合法之外还有哪些业务门禁
apps/api/study_agent/application/tutoring.py	effective_intent / TutorService.final_response	内容意图与最终有效证据如何重新确认
examples/learning/agent_tool_loop.py	run_demo	固定模型和工具结果怎样验证真实 Agent 图
apps/api/tests/test_tutor_acceptance.py	test_a01 / test_a07 / test_a09	工具循环、伪造引用和预算边界有哪些验收证据

下一章：第06章用一个独立手写 StateGraph 拆开节点、条件边、reducer 和中断恢复；第07章再回到生产辅导的持久化与幂等。

主要参考：本地飞书 LangChain、Agent、RAG 与 LangGraph 课程的教学组织方式，以及当前项目源码。具体框架接口以锁定环境与官方文档为准。